APPLIED MATERIALS HACKATHON SUBMISSION

Drift-Sense

AI-Powered Navigation-Error Recovery for Wafer Inspection Tools

"Think in a systems way"

Team Sanchari

Amrita Vishwa Vidyapeetham, Bangalore

Balcha Parswanadh	Team Lead	venkataparswanadh@gmail.com	
Ch Sriya Bharathi	Member	sriyabharathichaluvadi@gmail.com	7981197476
M Padmaja	Member	mutyalapadmaja77@gmail.com	9440307686

Problem Statement: Navigation-Error Recovery

- A wafer inspection tool must revisit the exact same die site thousands of times a day, across hundreds of dies, with nanometre repeatability — otherwise measurements from different visits aren't comparable.
- Motion stages never repeat perfectly: thermal expansion, fab-floor vibration, and mechanical slack accumulate as drift between visits.
- Every die on a wafer carries the identical repeating layout, so the tool cannot tell from the image alone that it landed a few pixels off — the wrong site looks almost exactly like the right one.
- Classical template matching breaks down exactly here: in DRAM or FinFET arrays, hundreds of near-identical features sit in one frame, with no principled way to pick the right one.

The real question

Not "does this look like the target?"

but "which of many near-identical targets is the correct one?"

Solving this reliably and fast keeps an inspection tool's measurements trustworthy across a wafer, across tools, and across time.

Idea Description

Architecture: DRAM-style periodic word-line / bit-line / via array as the primary narrative — generator also supports FinFET-style parallel-fin / gate structures (both are in Applied Materials' hidden test set).

Localization: classical computer vision — multi-scale / rotation normalized cross-correlation (OpenCV matchTemplate, TM_CCOEFF_NORMED) — not deep learning. Zero training data, fully deterministic, and easy to explain when it fails, which matters directly for the 10% explainability score.

Why this beats simple template matching on periodic layouts

1

Known ratio, not a guess

The 10x zoom ratio is exact by the problem's own definition. We sweep a narrow scale band around it instead of a blind multi-scale pyramid search — faster and less prone to locking onto the wrong scale.

2

Periodicity isn't ignored

Naive matching returns one best-scoring location and calls it done. In a DRAM/FinFET array dozens of locations score almost identically — we run non-max suppression to surface all strong candidate peaks first.

3

AM's own tie-break rule

Among near-tied peaks, we return whichever is closest to the search image's center — exactly Applied Materials' stated disambiguation rule — turning ambiguity into documented, explainable behavior.

Proposed Solution

Dataset Generator

One shared native canvas → native-res crop = Reference (1nm/px); larger randomly-placed 10x crop, INTER_AREA-downsampled → Search (10nm/px). True center recorded exactly from crop arithmetic.

Noise:
independent Poisson (shot) + Gaussian (read) noise per image, drawn separately — search image intentionally noisier.

Degradation:
independent per-image rotation, Gaussian blur (beam PSF), Sobel-gradient edge brightening (SEM secondary-electron contrast).

Site markers:
a locally-unique defect mark on most pairs (mirrors AM's own example) — a purely periodic site has no information to disambiguate from a single crop.

Localization Algorithm

1. CLAHE-normalize both images (counters independent capture contrast).
2. Sweep a narrow scale band around the known ~10x ratio + a small rotation band.
3. cv2.matchTemplate (normalized cross-correlation) at each combination; keep the best.
4. Non-max suppression on the best correlation surface → top local peaks.
5. Tie-break: among near-tied peaks, pick the one closest to the search image's center.
6. Output (x, y) + confidence + diagnostics (scale, rotation, ambiguity flag).

Ref + Search → CLAHE → Scale/Rot Sweep → NCC Match → NMS Peaks → Center Tie-break → **(x, y)**

Innovation & Uniqueness

Known-ratio search, not blind multi-scale

We treat the 10x relationship as a known physical constraint, not an unknown to search for — a narrow, fast, well-targeted scale band instead of a wide, slow, error-prone one.

Periodicity as a first-class citizen

Rather than reporting the argmax and hoping, we explicitly detect near-ties and resolve them with AM's own rule — the hardest case becomes documented, explainable behavior, not a silent wrong answer.

Realistic, non-degenerate synthetic data

Structural parameters are randomized per pair within literature-informed bounds, and pitch is deliberately kept large enough to survive the mandatory 10x downsample without aliasing into unresolvable texture.

No training data or GPU dependency

The whole pipeline runs on CPU in ~1.2s per pair, with zero risk of overfitting to synthetic-data artifacts that wouldn't generalize to Applied Materials' held-out test set.

Results

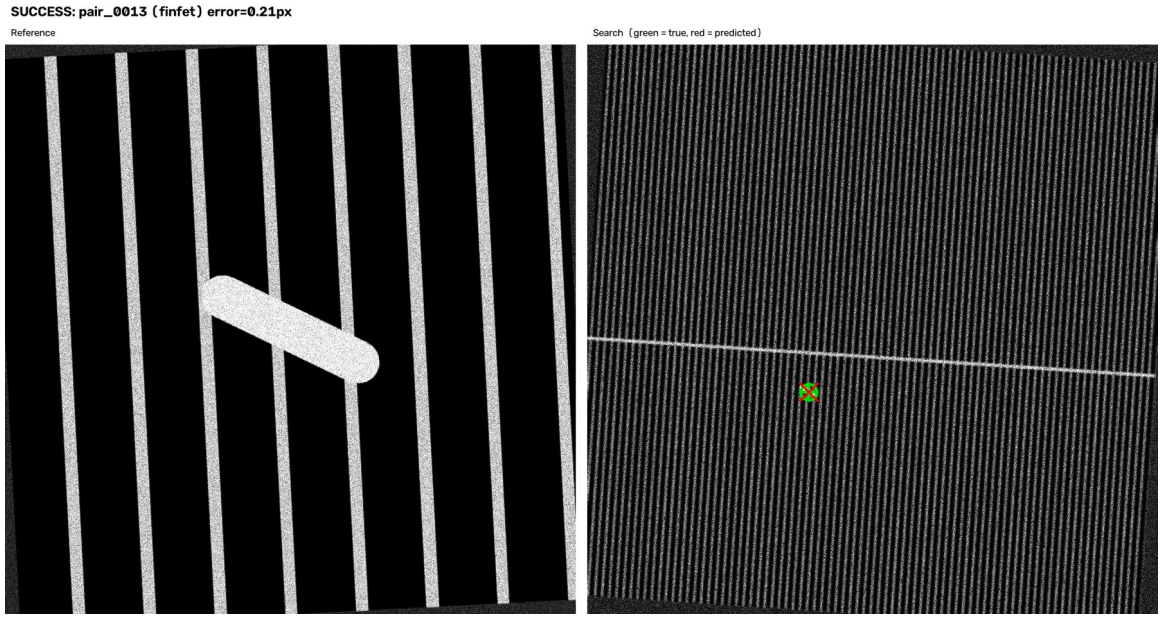
86.7%
within 3px overall
(30-pair self-eval)

100%
within 3px on sites
with a unique marker

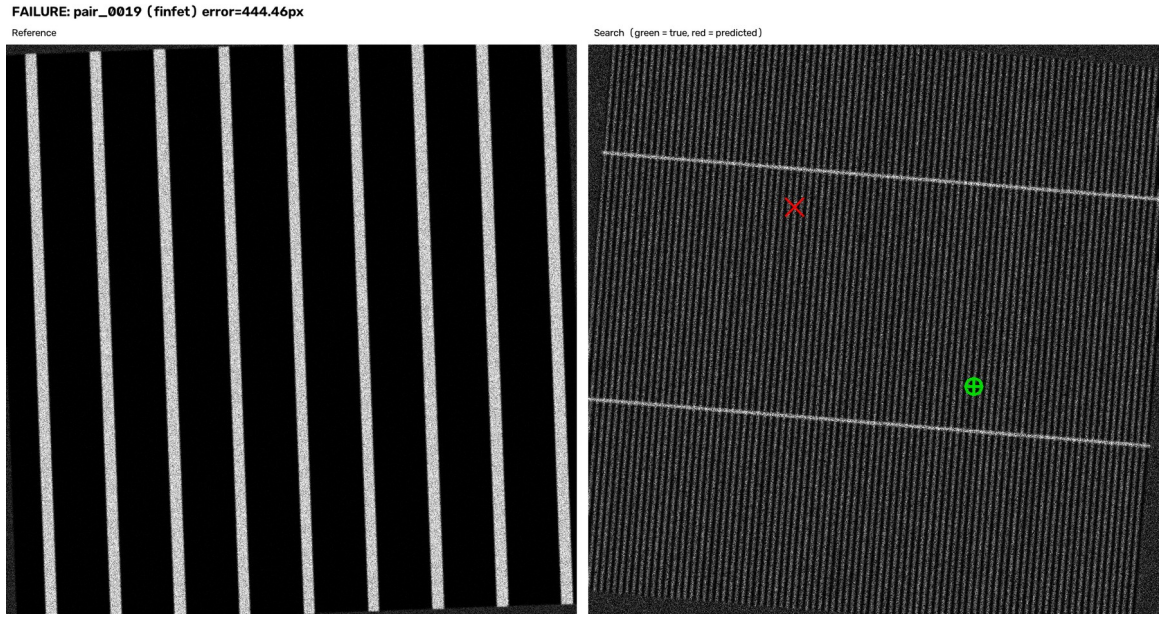
0.47px
median error, overall
(bimodal: near-perfect or honest fail)

~1.25s
mean inference time
per 1000×1000 pair (CPU)

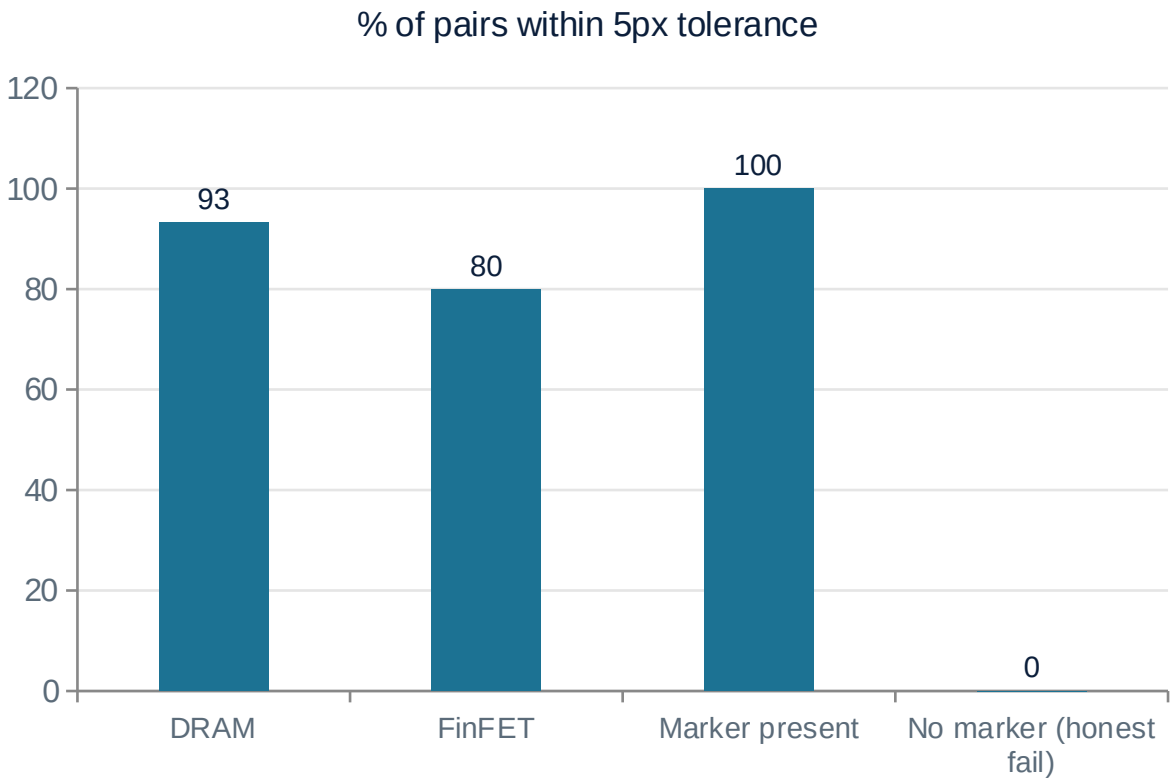
SUCCESS — pair_0013 (finfet), error = 0.21px



HONEST FAILURE — pair_0019 (finfet), error = 444.5px



Results — Breakdown & Root Cause



Root cause of the honest failure

pair_0019 (finfet) had no locally-unique site marker — the reference crop sits in a purely periodic region, so many lattice repeats are statistically indistinguishable from the true site.

Applied Materials' own “closest to center” disambiguation rule is applied, but it only recovers the correct site when the true site happens to be the center-closest repeat. **In a genuinely periodic, marker-free region, no single-crop image content can ever distinguish the true site from its lattice neighbors — a fundamental information-theoretic limit of template matching on periodic layouts, not a bug in the search strategy.**

Technology & Feasibility

Tech stack

Python 3, NumPy, OpenCV (opencv-python-headless), Pillow, SciPy, scikit-image, Matplotlib

Dataset generation

~25-30s for 32 image pairs (both architectures) on CPU

Model size

N/A — classical algorithm, no trained weights to ship

Hardware

Local machine, NVIDIA RTX 4070 Laptop GPU available but NOT used — classical CPU-only pipeline, no training required

Inference time / pair

Mean 1.25s, median 1.24s, p95 1.33s on a 1000×1000 pair (CPU)

Bonus track

RGB / optical-microscope generalization not yet attempted — core SEM grayscale pipeline completed first per rubric guidance

GitHub & Demo

GitHub Repository (public)

github.com/Parswanadh/sanchari-drift-sense

Contains: dataset_generator.py, localize.py, eval_selftest.py, citations.md, requirements.txt (pip freeze), README.md with full setup instructions.

Demo Video

Optional — to be added: a short recording of localize.py running on a sample pair, showing the printed (x, y) output.

References

Full reference list with verified public sources lives in citations.md in the repository root (six categories, 2-3 sources each).

1. Poisson (shot) + Gaussian (read) sensor noise model for SEM imaging
2. SEM secondary-electron edge brightening / edge contrast effect
3. Gaussian blur as a model of electron-beam spot size / PSF
4. Motion-stage drift, vibration, and thermal positioning error in wafer inspection tools
5. DRAM memory cell array structure (word-line / bit-line / via grid)
6. FinFET transistor structure (parallel fin arrays + gate structures)

See citations.md for full titles, authors, venues, and verified URLs.